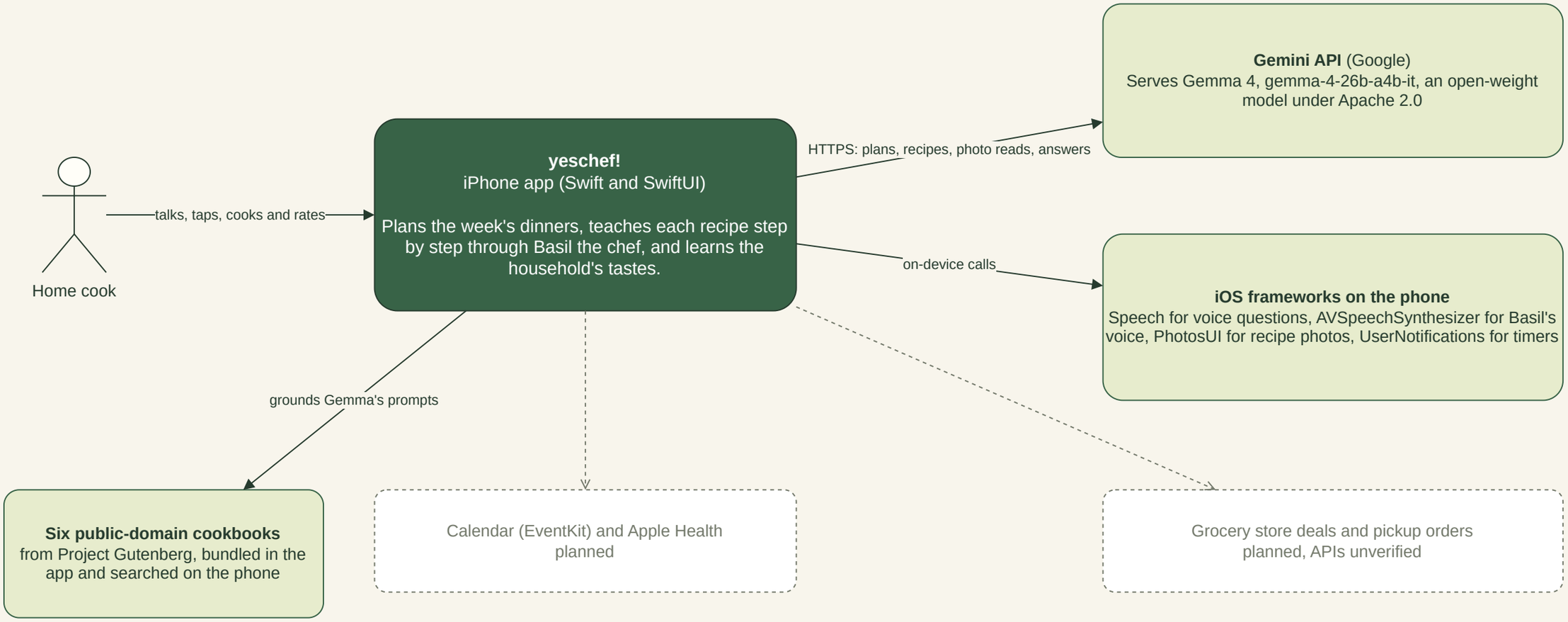
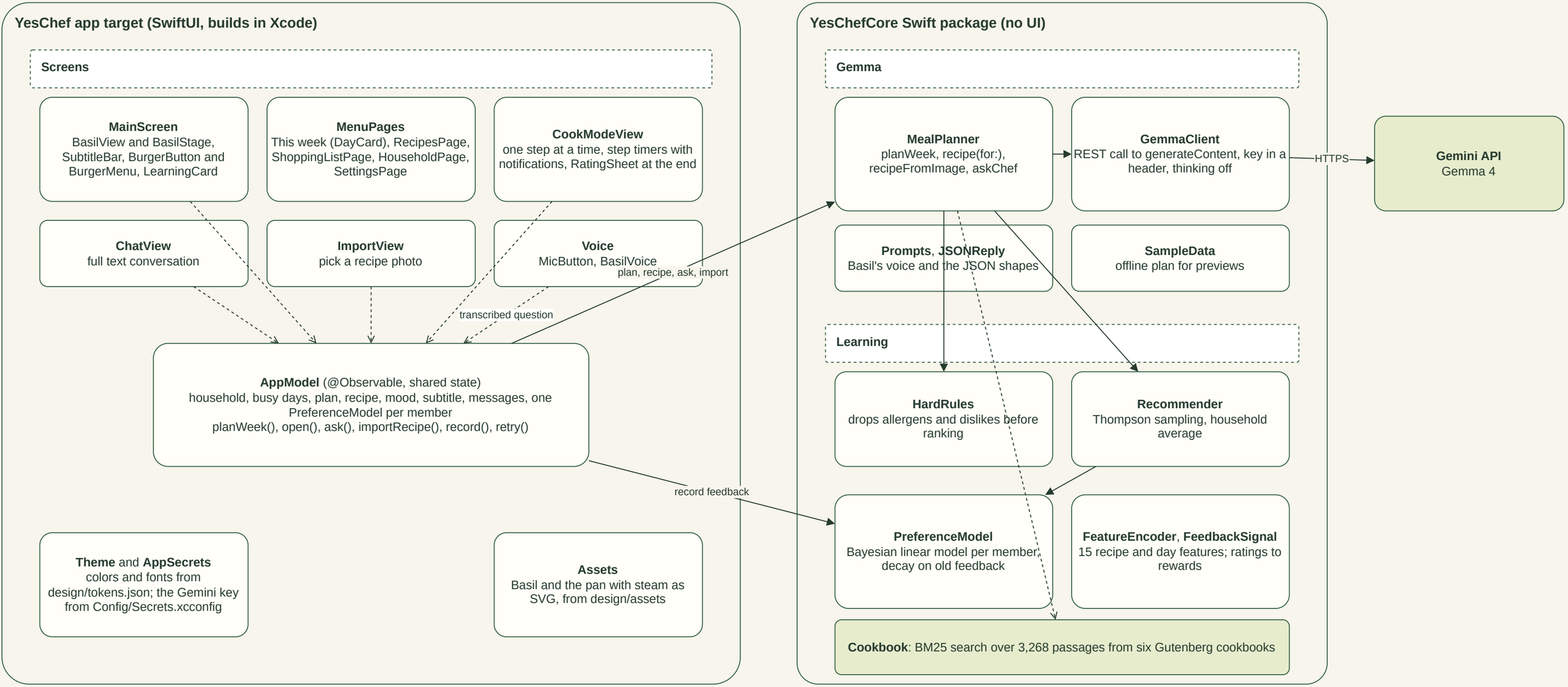
# 1. System context

Who and what yeschef! talks to. Solid lines are in the demo; dashed boxes are planned for after the hackathon. Everything not shown stays on the phone, and the demo keeps its data in memory only.



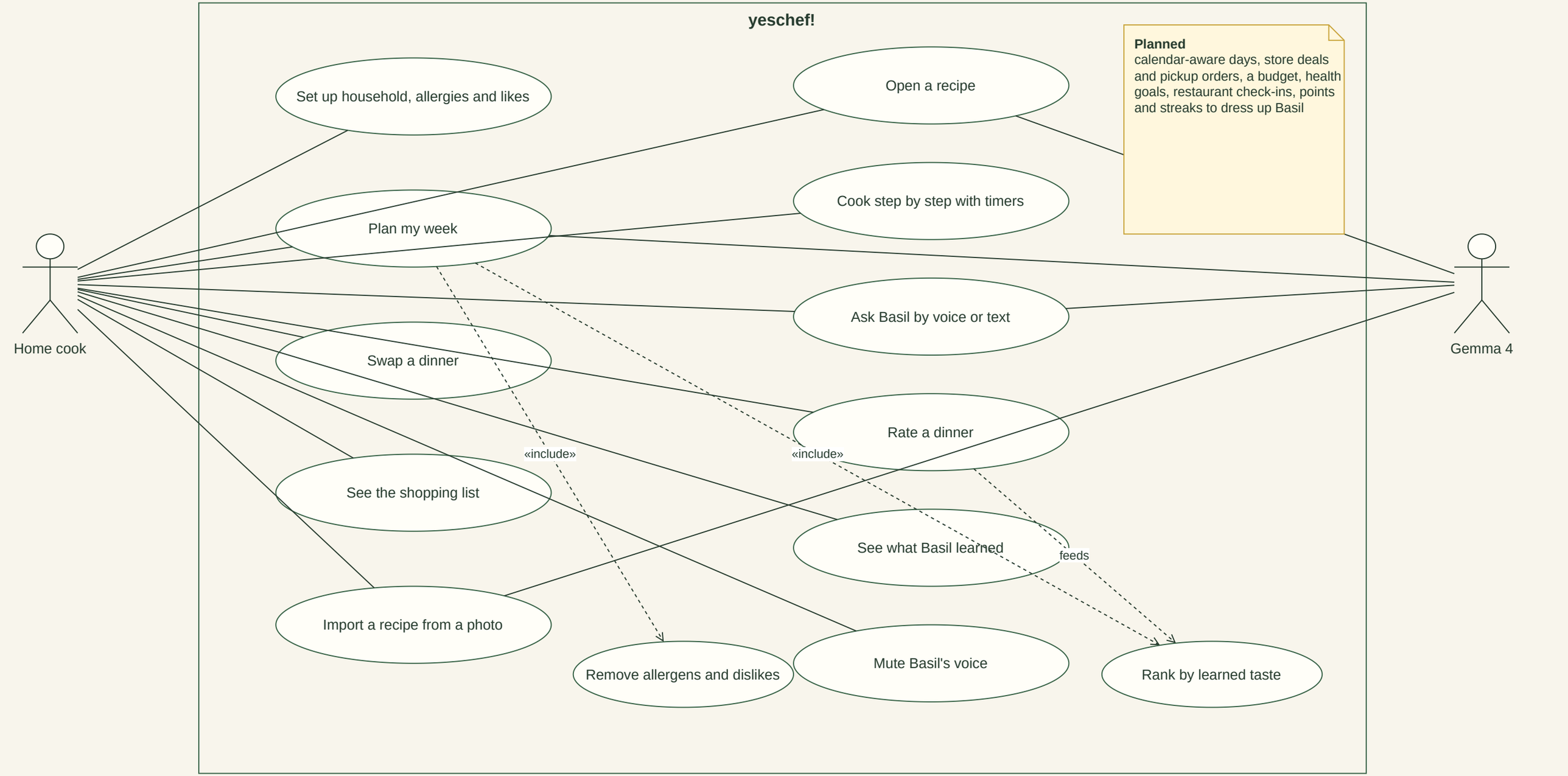
# 2. Containers and components

The app target holds the screens and anything that needs an Apple-only framework. YesChefCore holds the Gemma calls and the learning, has no UI, and builds and tests on the Mac and on Linux. Screens never call Gemma themselves; they go through AppModel.



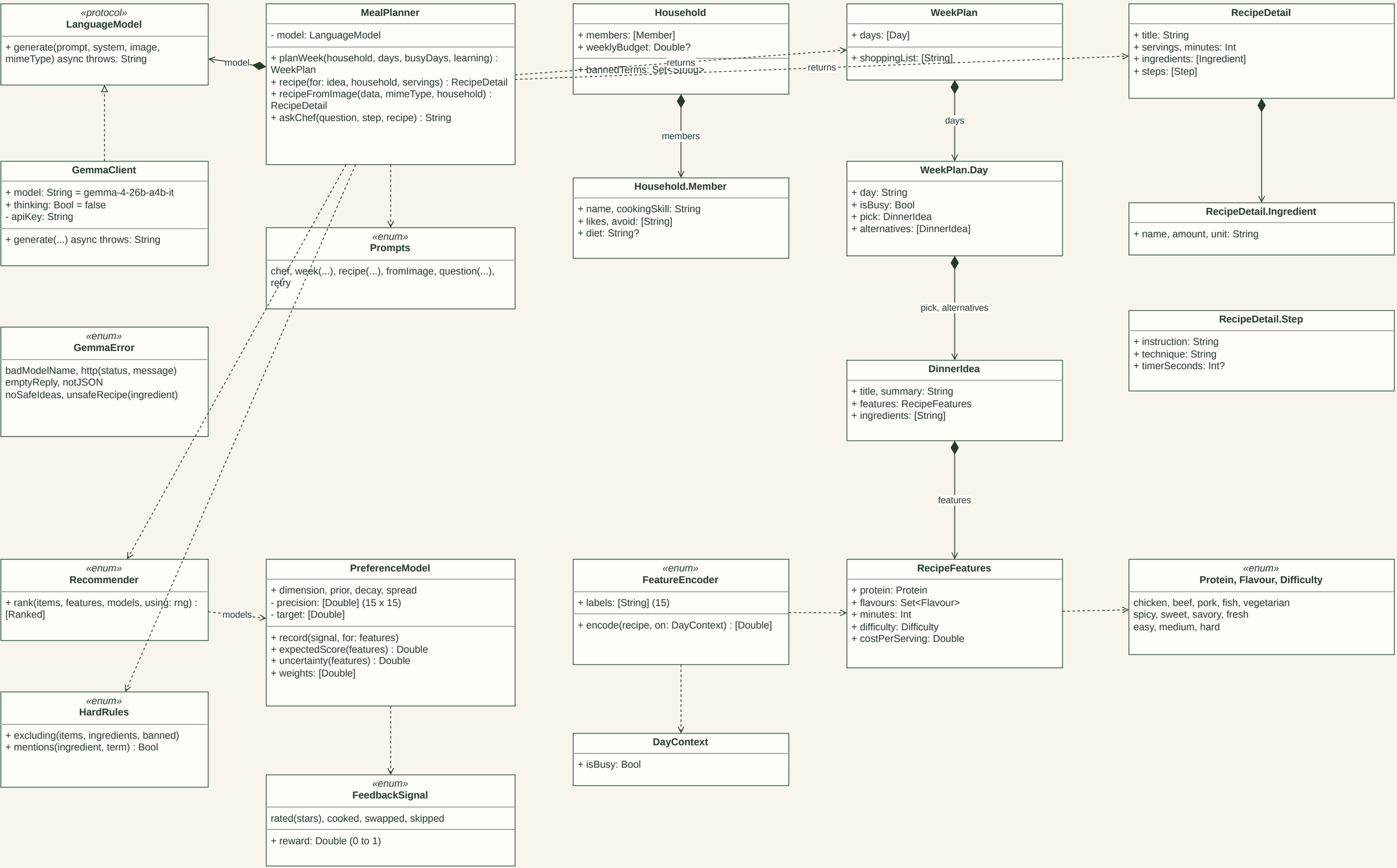
# 3. Use cases

What a home cook can do in the demo, and which of those call Gemma 4. The dashed include lines mark steps every plan goes through. Planned features are listed in the note.



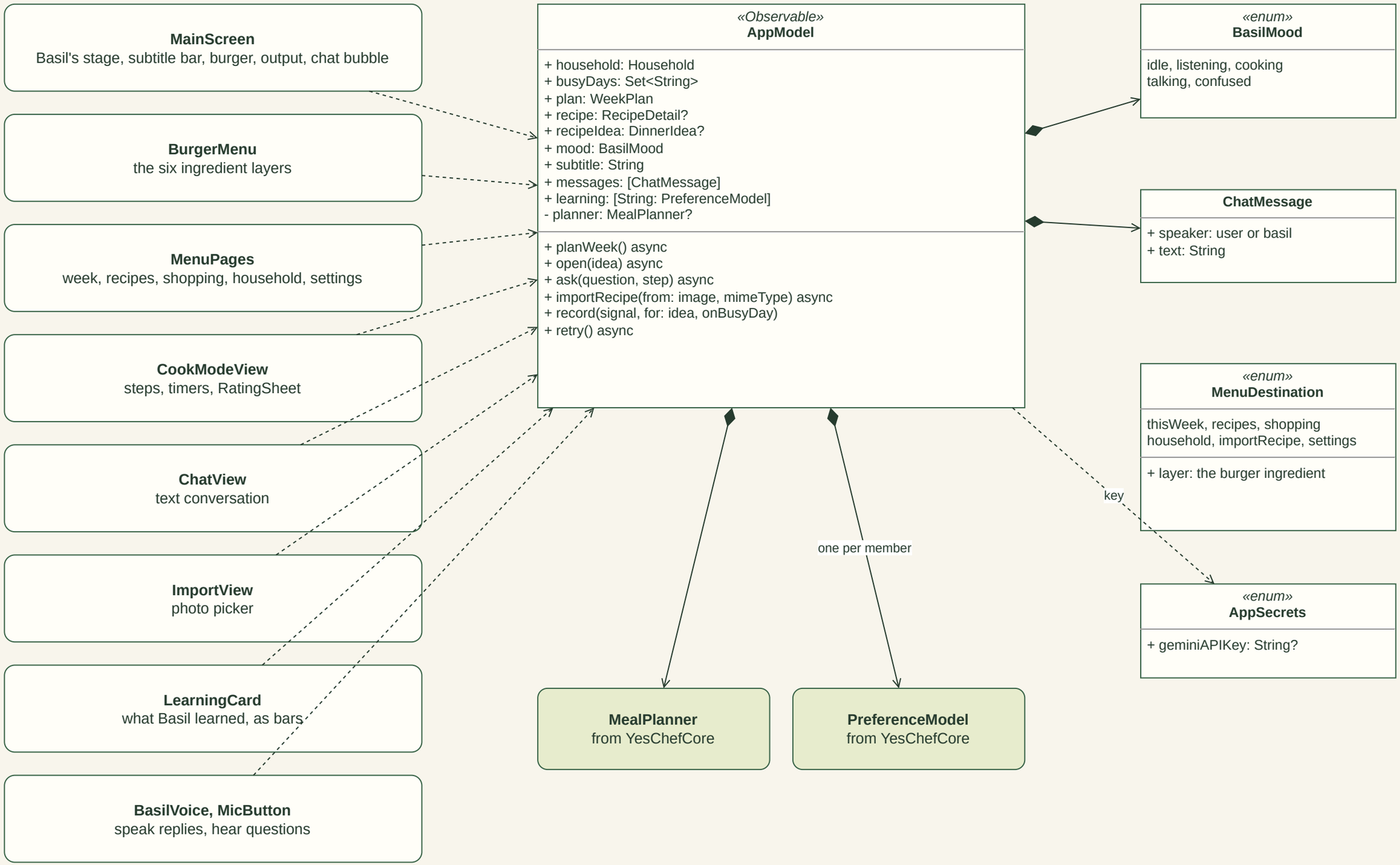
# 4. Class diagram: YesChefCore

The types the app builds on. All of them are value types except GemmaClient's network call, which keeps them testable on Linux. Filled diamonds mean one type holds the other; dashed arrows mean one uses the other.



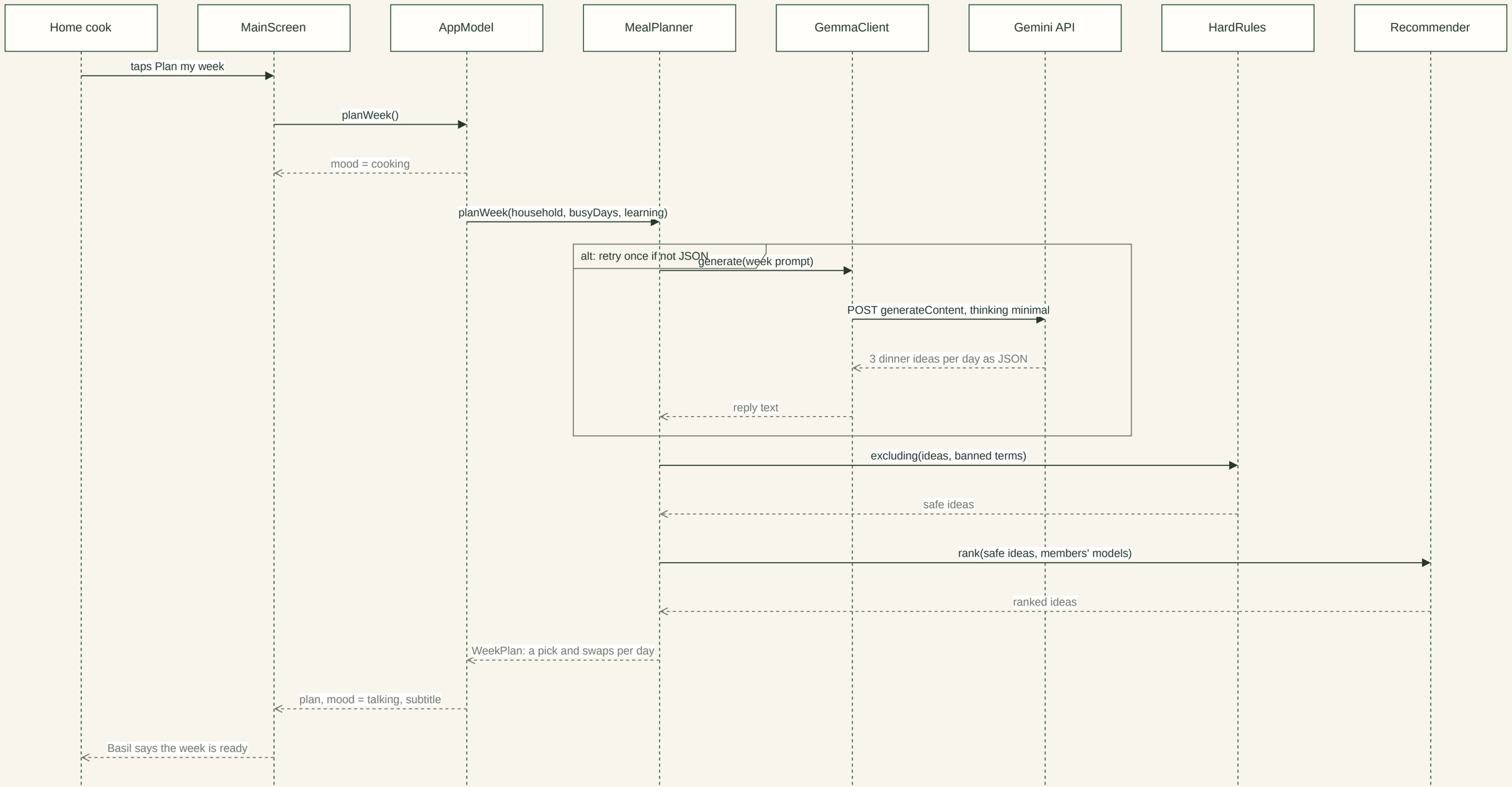
# 5. Class diagram: the app target

AppModel is the one shared object. Every screen reads it from the SwiftUI environment and calls its methods, and it is the only place that talks to YesChefCore.



# 6. Sequence: planning the week

What happens between tapping Plan my week and seeing the plan. Solid arrows are calls and dashed arrows are replies. Basil does his cooking animation for the whole wait.

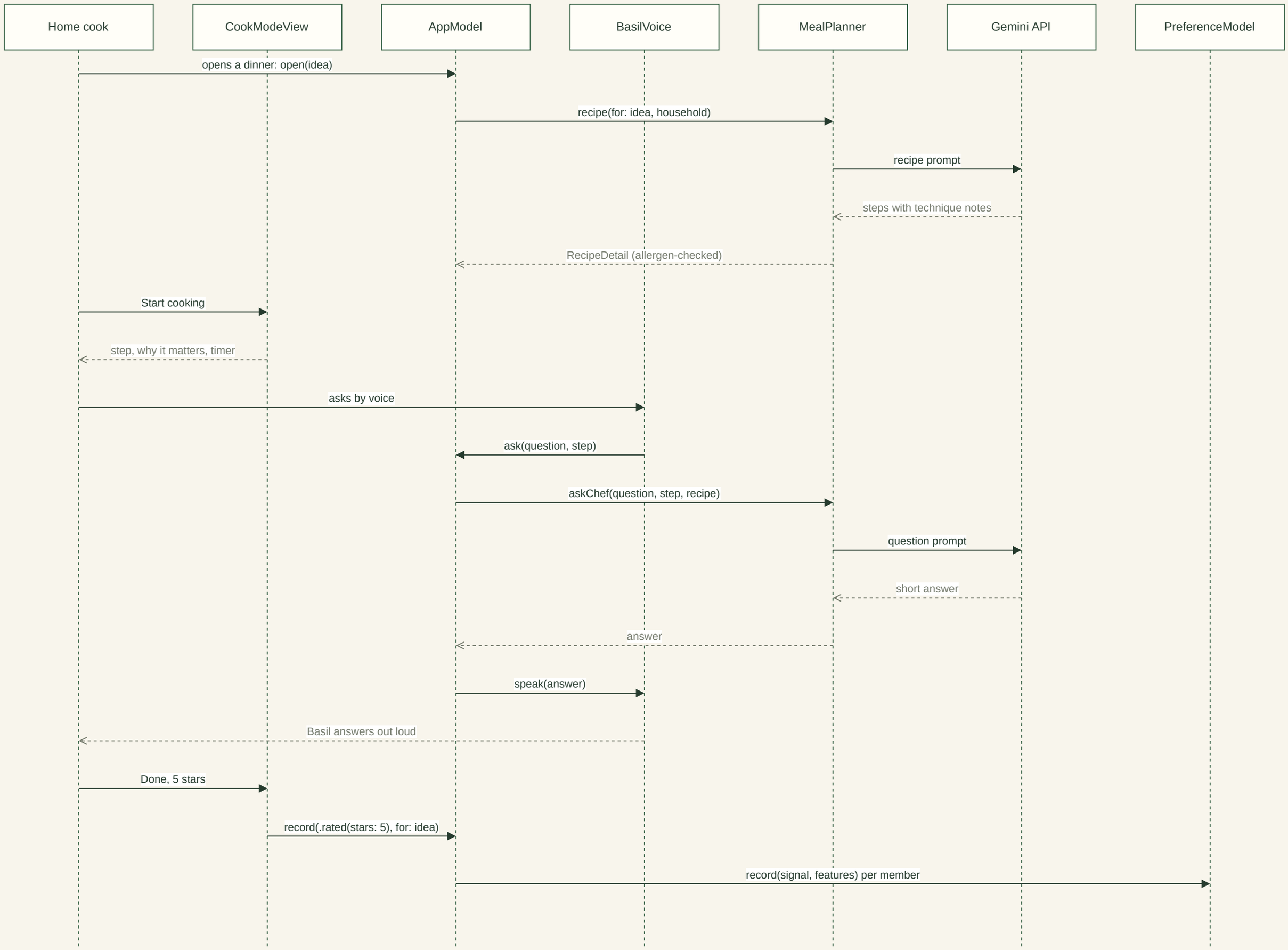



If the reply still isn't JSON after one retry, or no idea is safe, AppModel sets mood = confused and offers a retry.

Before it calls GemmaClient, MealPlanner searches the bundled Gutenberg cookbooks for the household's tastes, drops passages that name a banned ingredient, and puts the best six in the prompt with their books. Gemma names the book in each idea it based on one.

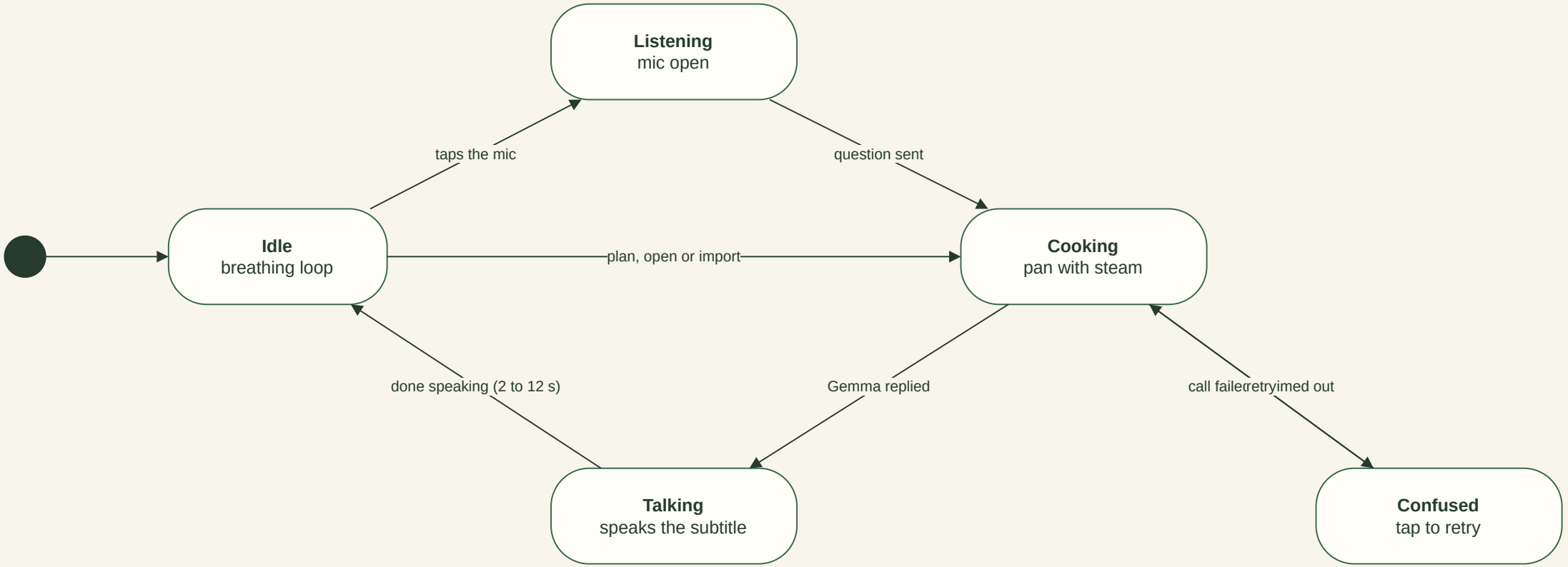
# 7. Sequence: cooking a recipe, asking Basil and rating it

Opening a recipe fetches the full steps, cooking runs one step at a time, a voice question goes through Gemma, and the rating at the end updates every member's preference model.



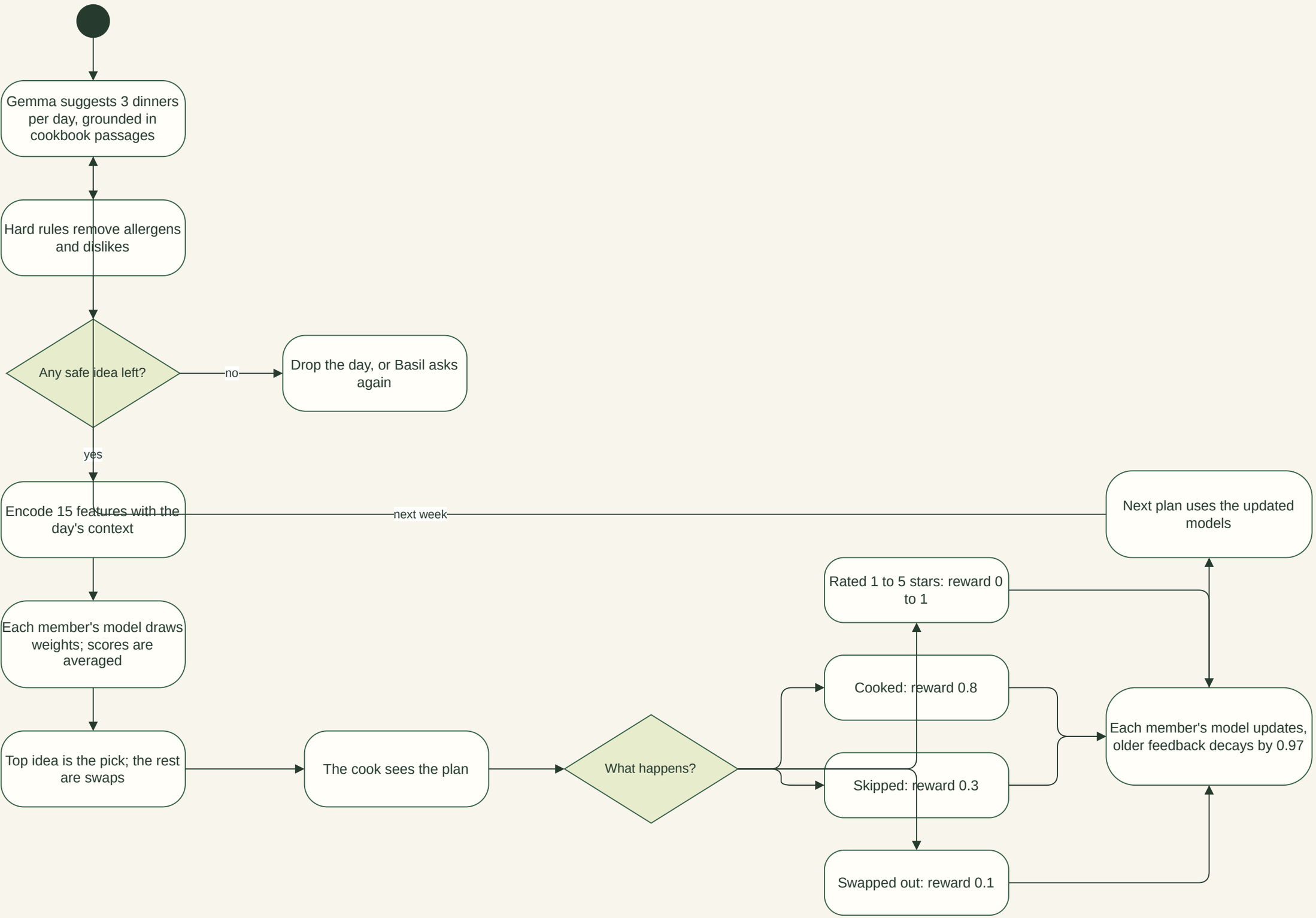
# 8. State machine: Basil's moods

Basil's mood drives his animation and the subtitle bar. Every Gemma call goes through cooking, and every answer ends back at idle.



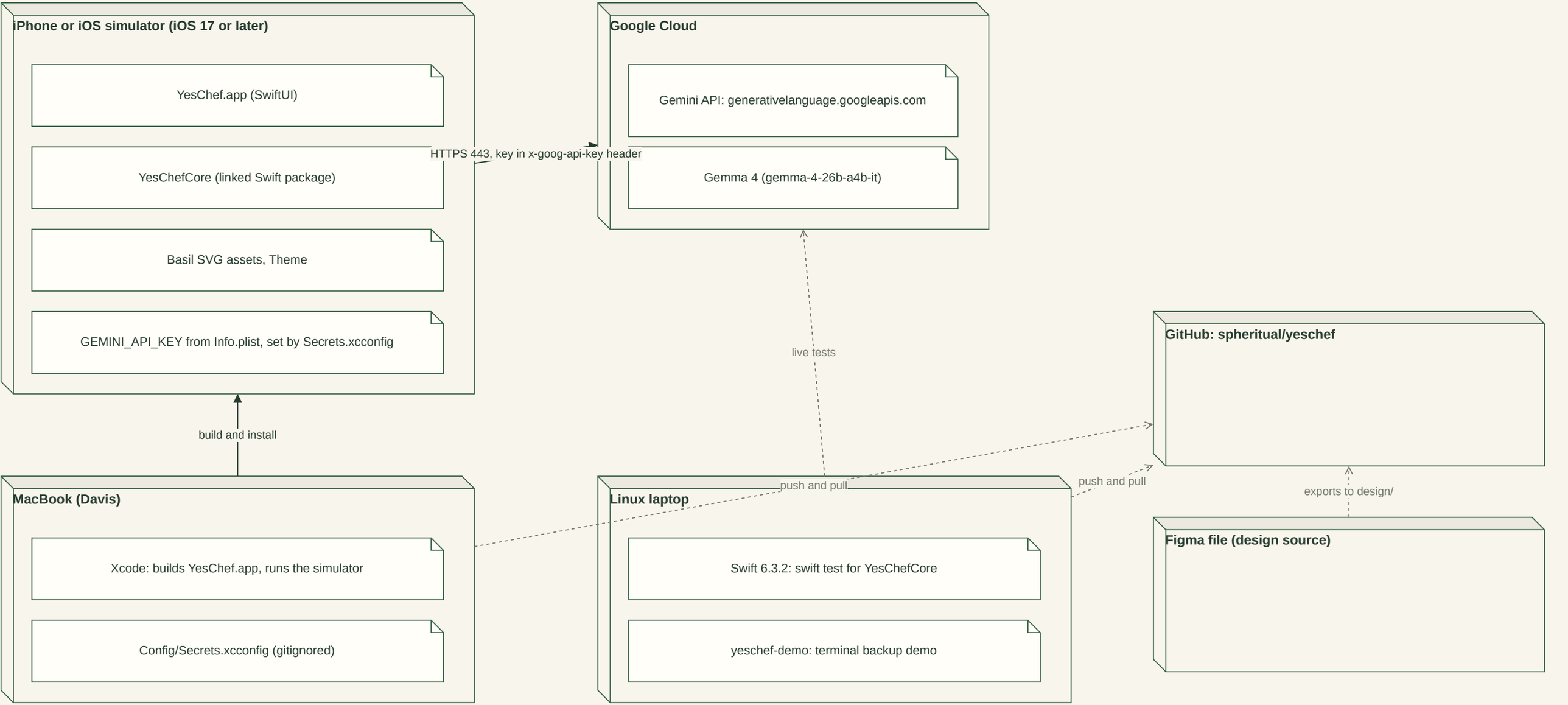
# 9. Activity diagram: how Basil learns

The feedback loop from .plans/feedback-learning-spec.md. Allergy rules always run before the model, and every rating, cook, swap or skip changes the next plan.



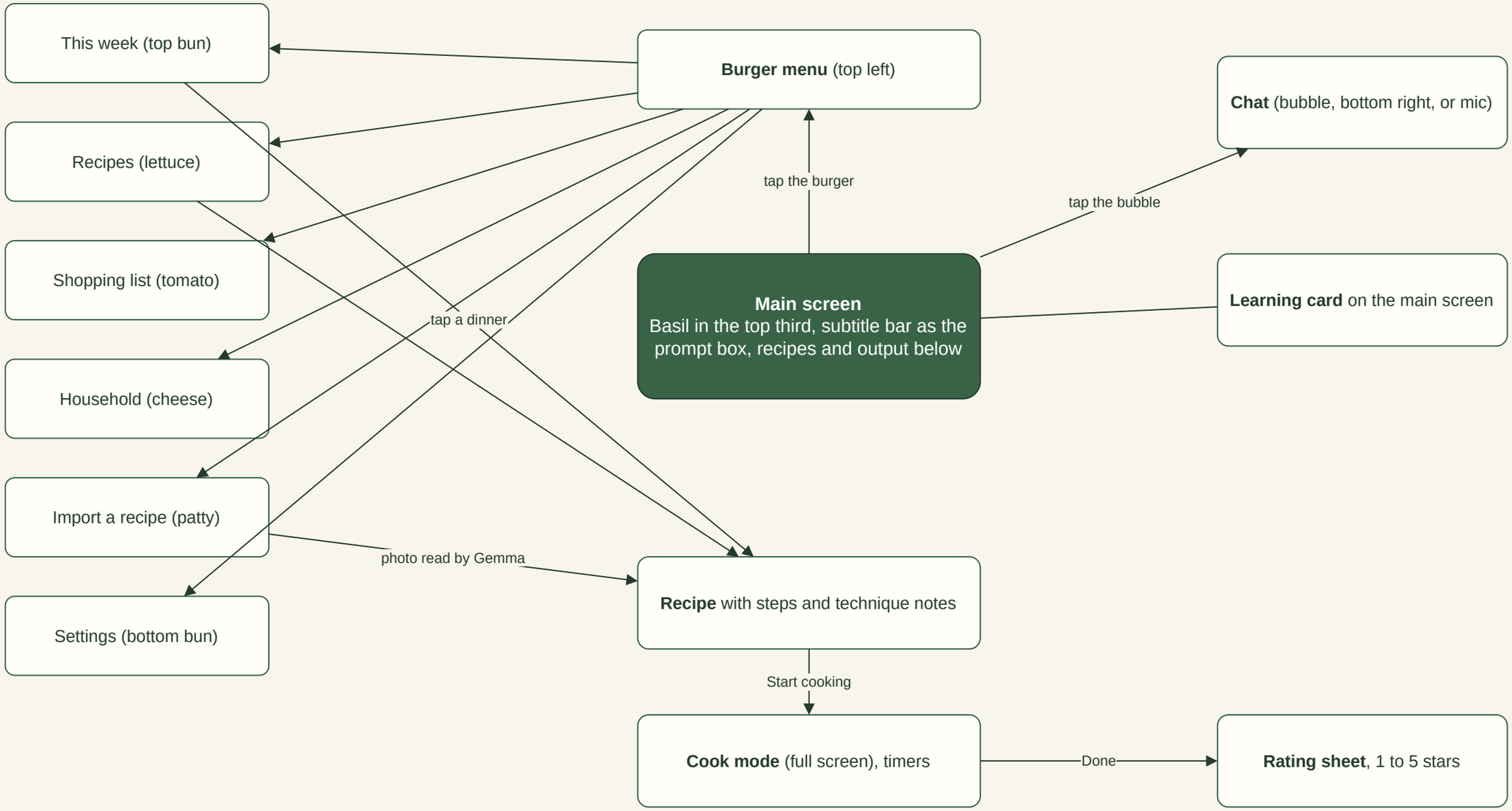
# 10. Deployment

Where each piece runs during the hackathon. The Gemini key lives only in the gitignored Config/Secrets.xcconfig on the Mac and in the laptop's keyring; it never goes in the repo.



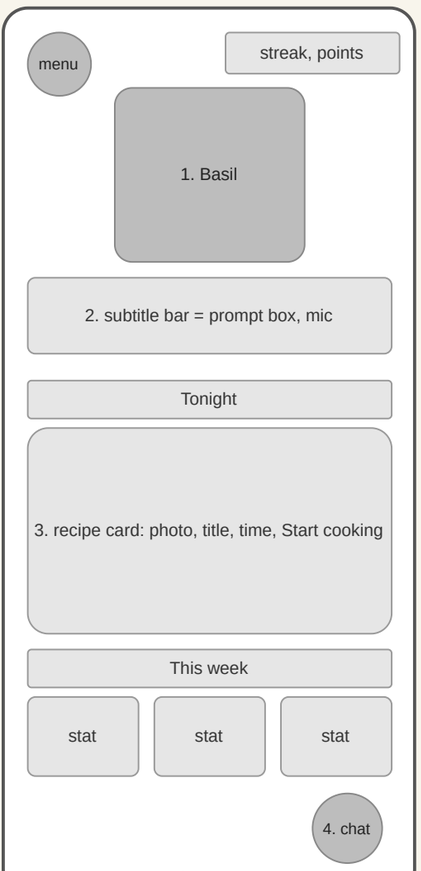
# 11. Navigation map

Every screen and how the cook reaches it. The app is one main screen; everything else opens on top of it as a sheet, except cook mode, which covers the full screen.

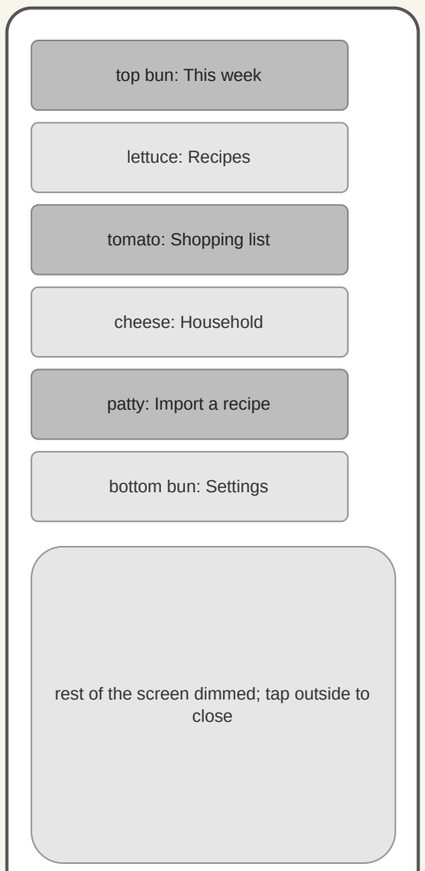


# 12. Low-fidelity wireframes

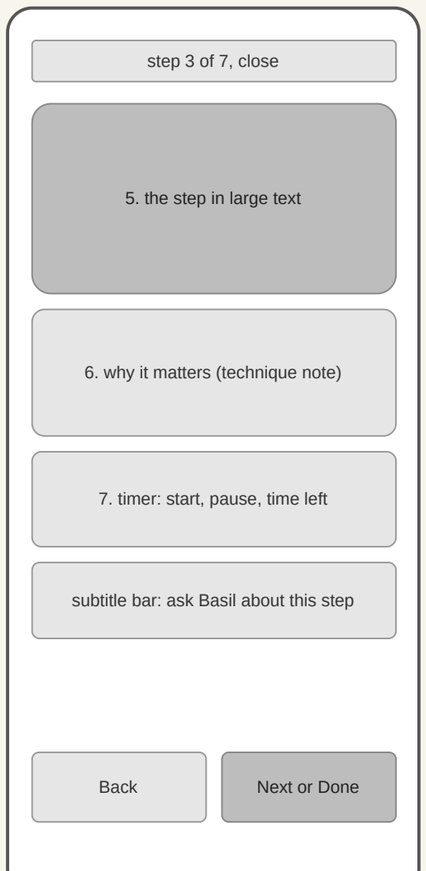
Layout only, in grey, before color. The numbered notes say what each block does. The Figma frames and design/screens show the finished look.




**Main screen**



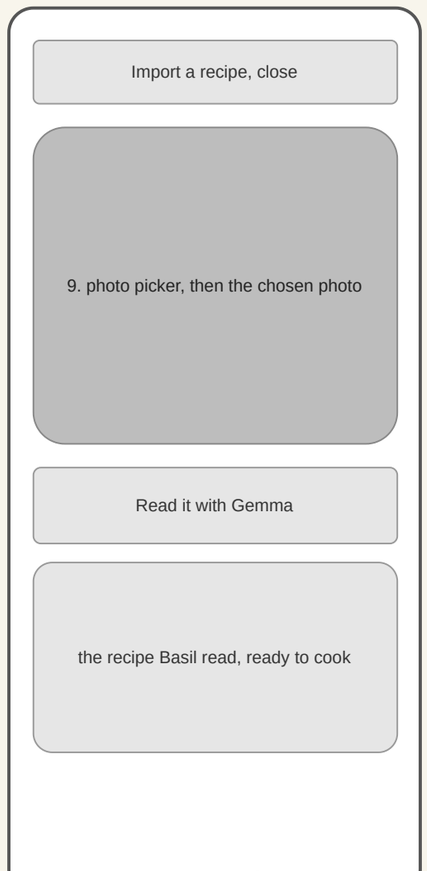

**Burger menu open**




**Cook mode**




**Chat**




**Import a recipe**

**Notes**

1. Basil changes mood with what's happening (tab 8). 2. The subtitle bar shows his last line and takes typed questions. 3. The output area shows tonight's dinner, the week or the current recipe. 4. The chat bubble opens the full conversation. 5 to 7. Cook mode keeps one step on screen, with the reason behind it and its own timer. 8. Typed and spoken questions go to the same askChef call. 9. Photo import is the multimodal Gemma call, and the recipe it returns is checked for allergens before it's shown.

# 13. File structure

What each folder in the repo holds. gummi-main is another project's code that a teammate committed as reference; it isn't part of yeschef!.

```
yeschef/
  README.md                  what it is, the model and its license, how to run
  LICENSE                    Apache 2.0
  CLAUDE.md                  working notes for the coding agents
  .mcp.json                  Figma's MCP server for the agents
  YesChef.xcodeproj/         the Xcode project
  Config/YesChef.xcconfig    build settings; includes Secrets.xcconfig (gitignored)
  YesChef/                   the iPhone app target
    YesChefApp.swift, ContentView.swift, AppModel.swift, AppSecrets.swift
    Main/                    MainScreen, BasilView, BurgerMenu, MenuPages
    Cook/                    CookModeView with timers and the rating sheet
    Chat/                    ChatView
    Import/                  ImportView for recipe photos
    Learning/                LearningCard
    Voice/                   BasilVoice, MicButton
    Theme/                   Theme colors and fonts
    Assets.xcassets/         Basil and the pan with steam
  YesChefCore/               Swift package, no UI, builds on Mac and Linux
    Sources/YesChefCore/Gemma/      GemmaClient, MealPlanner, Prompts, SampleData
    Sources/YesChefCore/Learning/   PreferenceModel, Recommender, HardRules, FeatureEncoder
    Tests/YesChefCoreTests/         offline tests and live Gemini tests
  YesChefDemo/               terminal demo, the backup for the stage
  design/                    Figma exports: screens, SVG assets, tokens, screen code
  docs/                      requirements, architecture, and this design set
    design/                  yeschef.drawio, the feature sheet, the generator
  .plans/                    coding checklist and the feedback learning spec
  .claude/skills/            interview, swiftui-pro, swiftui-expert-skill
  gummi-main/                another project, committed as character reference
```